UCS3461 Foundations of Artificial Intelligence

INFORMED SEARCH

Semester 4, B.E. Computer Science and Engineering
12 January 2026

Sakaya Milton R
SSN College of Engineering

Uniform Cost Search

- Bag: Priority Queue
- Priority of a node n = shortest distance from start node to n
- Selects the node with the shortest path from the start node.
- Should select the node through which is the shortest path from the start node to a goal node!
- Weakness: Goal agnostic
  - Explores in every "direction"
  - Not in the direction of a goal

Heuristic Search

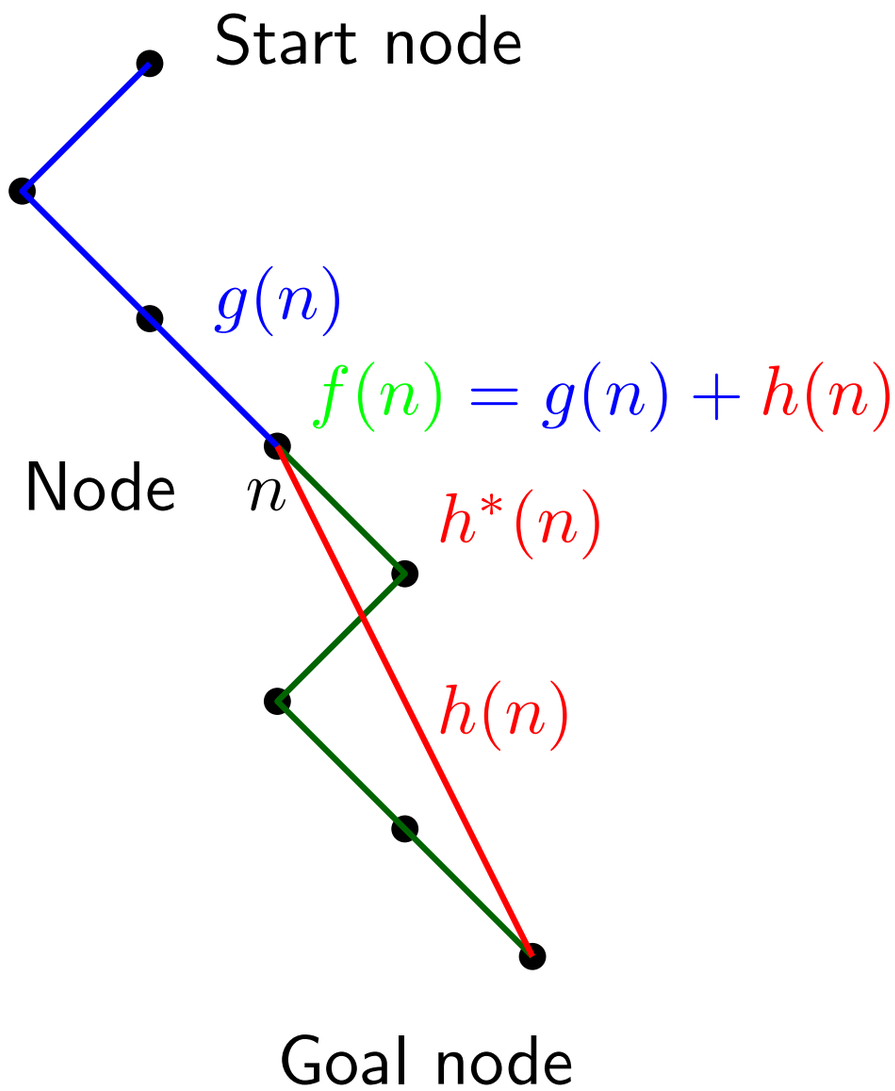
Heuristic Search

- Idea: don't ignore the goal when selecting paths.
- Often there is domain knowledge that can be used to guide the search toward the goal: heuristics.

Estimated Shortest Distance from Start Node to Goal Node

- Problem definition + Problem-specific knowledge
- $g(n)$: shortest distance from start node to node n – backward distance or past distance
- Heuristic function, $h(n)$: estimate of the distance $h^*(n)$ from n to the nearest goal node – forward distance or future distance
- Evaluation function, $f(n)$: estimated shortest distance from start node to a goal node via n

$$f(n) = g(n) + h(n)$$



Heuristic: Underestimate

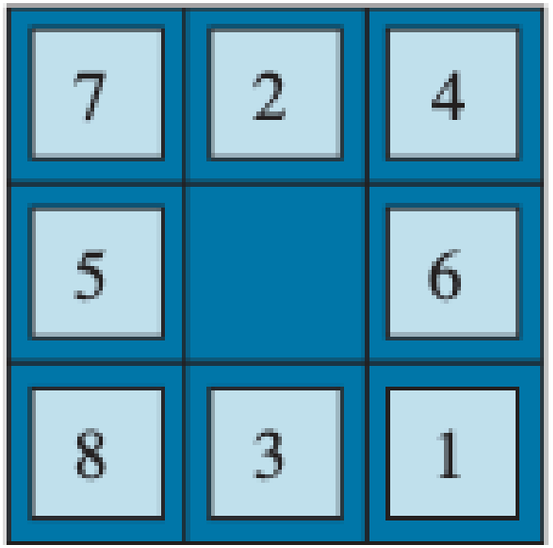
- A heuristic function $h(n)$ is a nonnegative estimate of the actual shortest distance $h^*(n)$ from node n to a goal node.
- Estimated distance $h(n)$ should be an underestimate of the actual shortest distance $h^*(n)$ (from n to a goal).

$$h(n) < h^*(n)$$

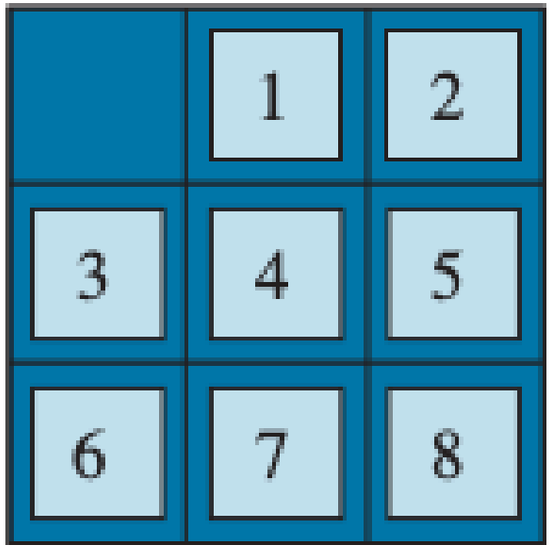
- An admissible heuristic is a heuristic function that is an underestimate of the actual shortest distance to a goal.
- $h(n)$ needs to be efficient to compute.

8-Puzzle Heuristic

- $h(v) =$ Number of misplaced tiles
  $h(v) = 8$



start state



goal state

Example Heuristic Functions

- If the nodes are points on a Euclidean plane and the cost is the distance, $h(n)$ can be the straight-line distance from n to the closest goal.
- If the nodes are locations and cost is time, we can use the distance to a goal divided by the maximum speed.
- A heuristic function can be found by solving a simpler (less constrained) version of the problem.

Greedy Best-First Search

- Heuristic $h(n)$: estimated distance to the nearest goal node
- Explore the node that "seems" nearest to a goal node

$$f(n) = h(n)$$

- Greedy best-first search selects a path on the frontier with minimal h-value.
- It treats the frontier as a priority queue ordered by h.

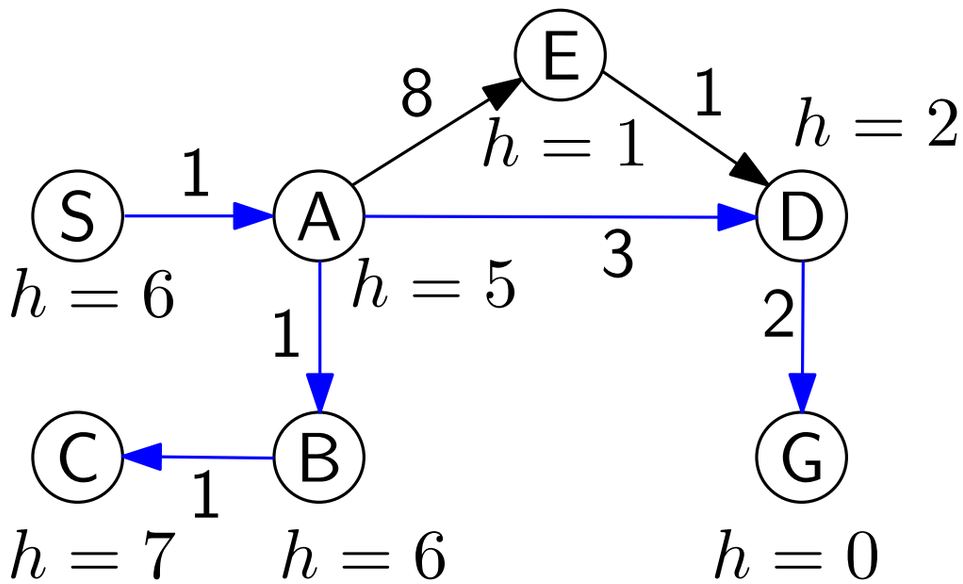
A* Search

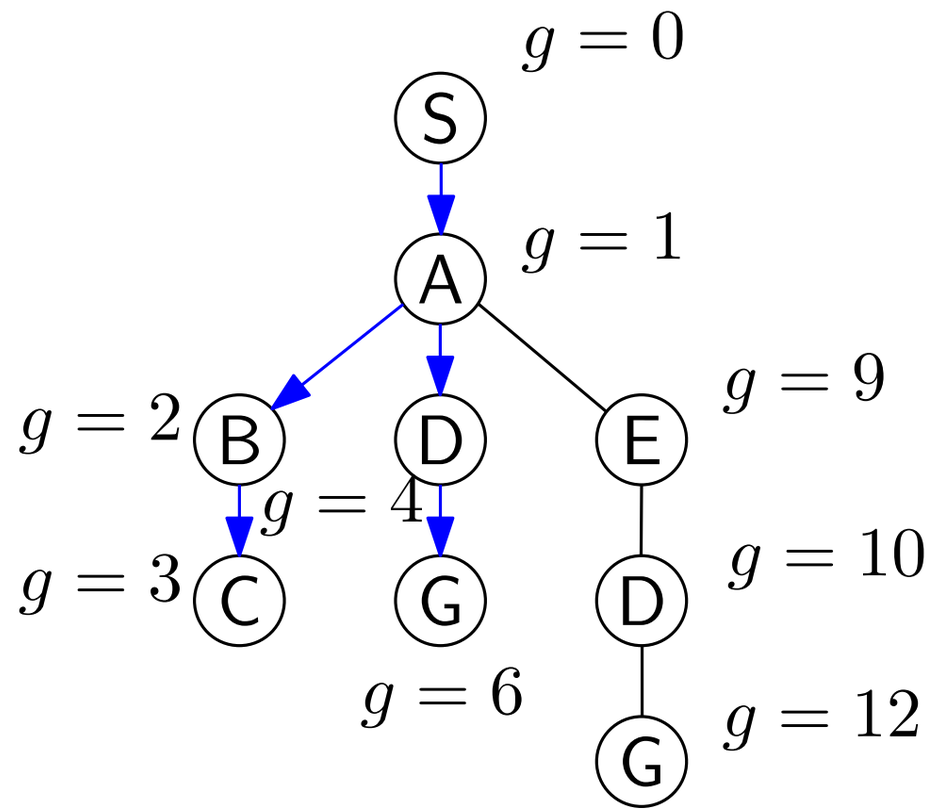
A* Search

- A* search uses both shortest distance $g(n)$ and heuristic value $h(n)$.
- $g(n)$ is the shortest distance from start node to n.
- $h(n)$ is the estimated distance from n to a goal.
- Let $f(n) = g(n) + h(n)$.
  $f(n)$ estimated shortest distance from start node to a goal via n.
- The frontier is a priority queue ordered by $f(n)$.
- It always selects the shortest estimated distance from the start to a goal node constrained to go via a frontier node.

A* Search Combines UCS and Greedy BFS

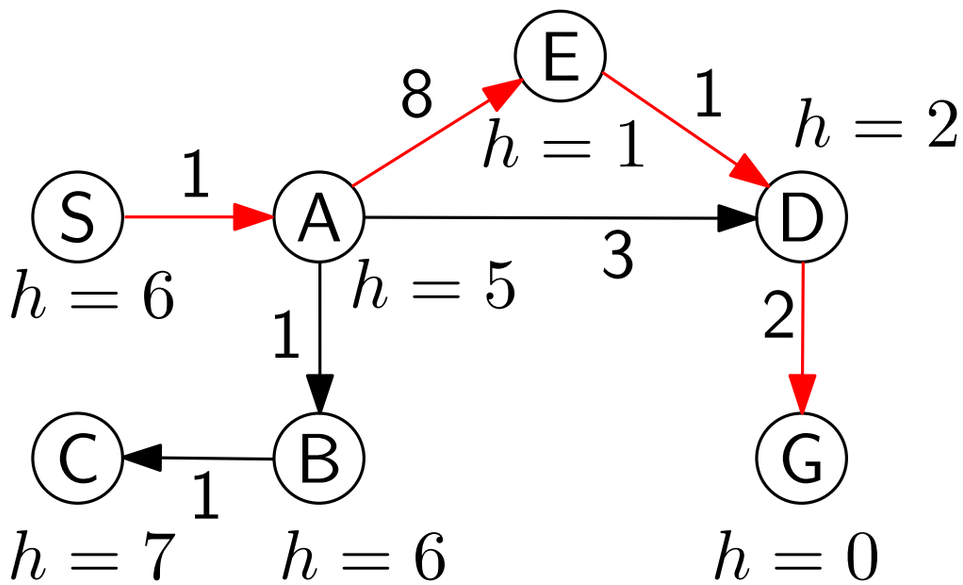
- UCS orders by backward path cost, $g(n)$

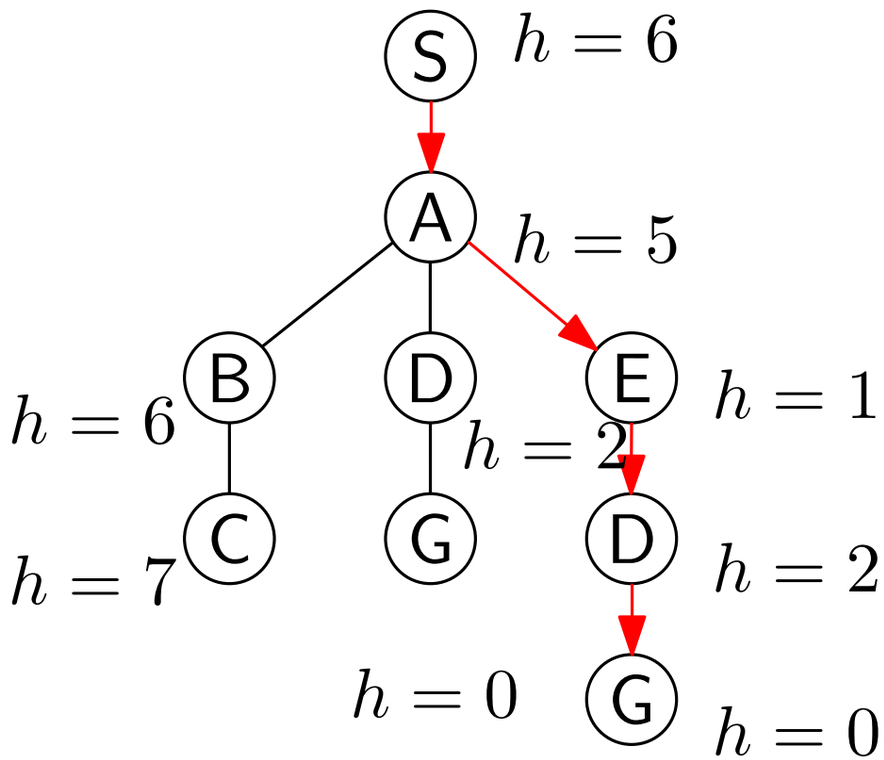




A* Search Combines UCS and Greedy BFS

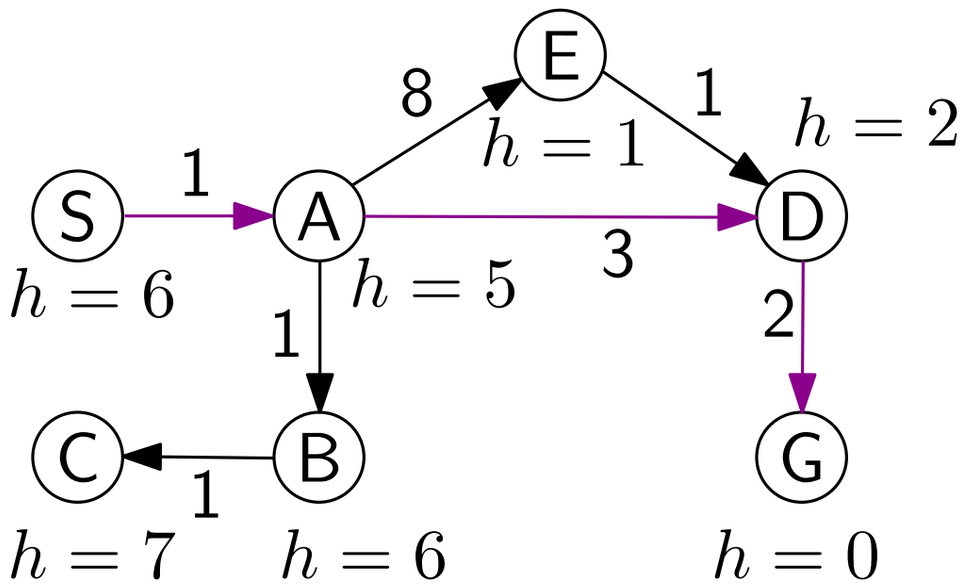
- UCS orders by backward path cost, $g(n)$
- Greedy Best-First Search orders by goal proximity, forward path cost, $h(n)$

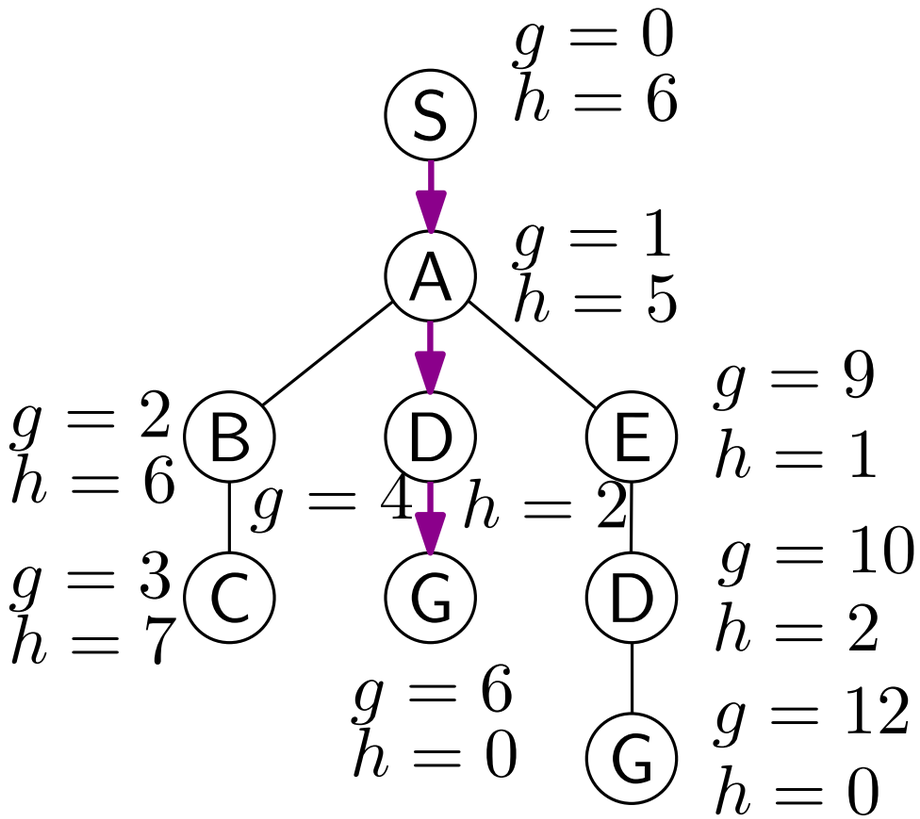




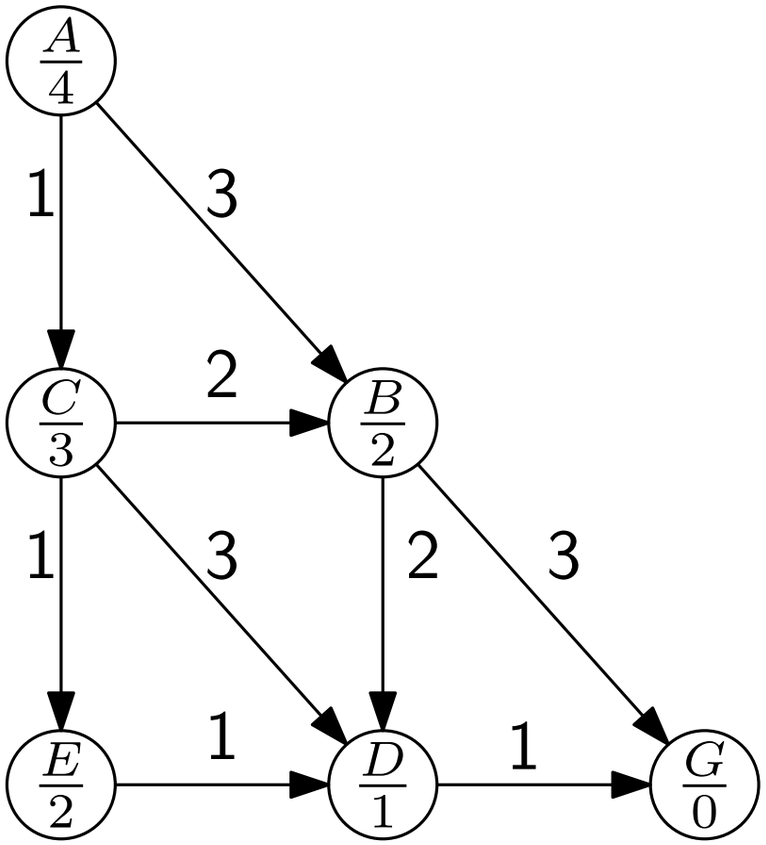
A* Search Combines UCS and Greedy BFS

- UCS orders by backward path cost, $g(n)$
- Greedy Best-First Search orders by goal proximity, forward path cost, $h(n)$
- A* orders by the sum: $f(n) = g(n) + h(n)$



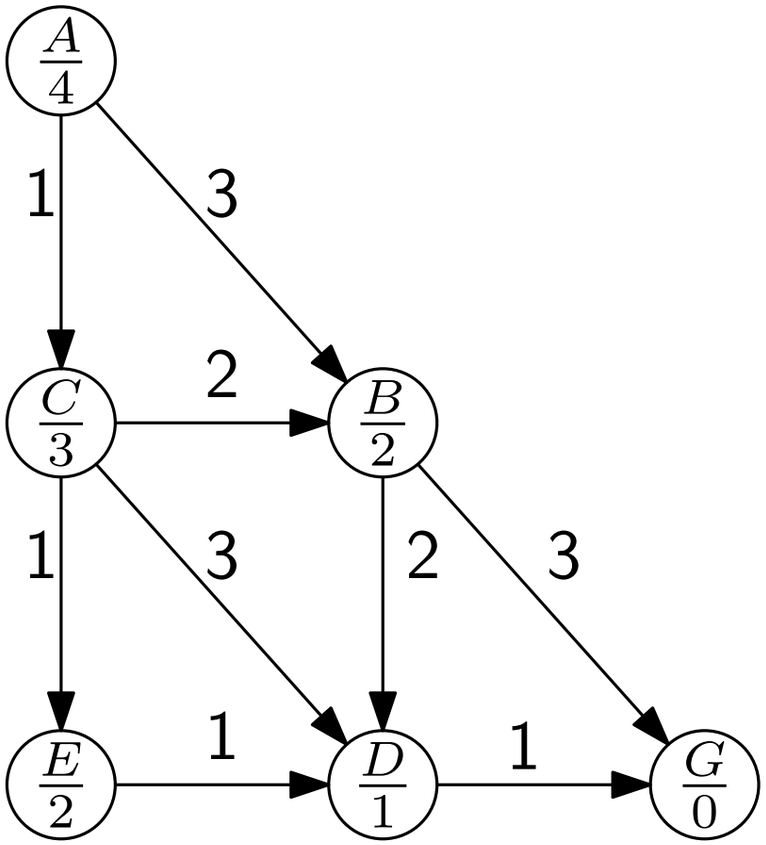


UCS Trace



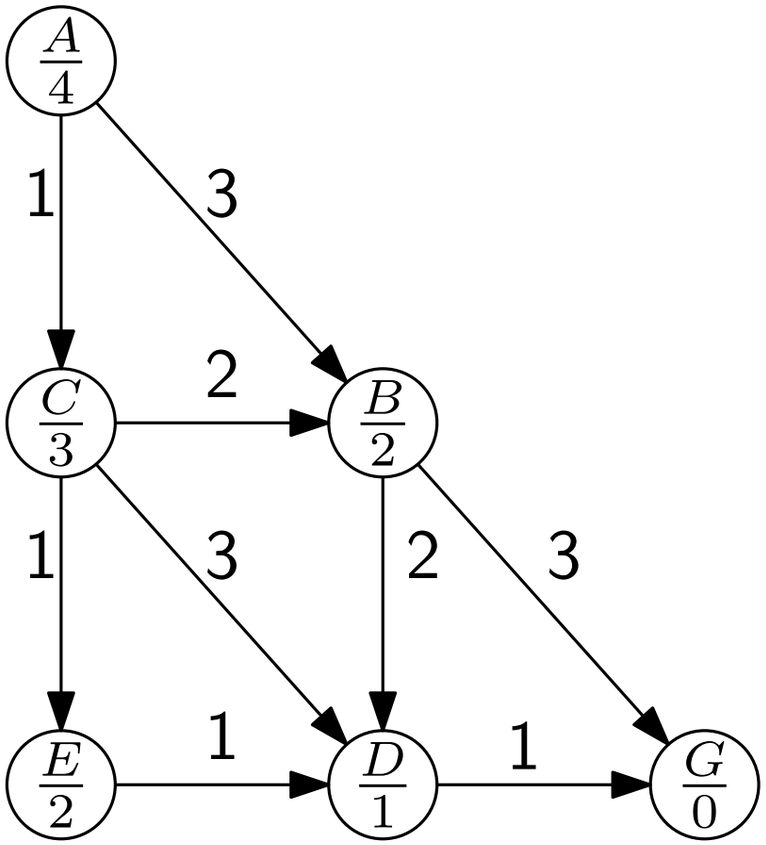
E	F
	A_0

UCS Trace



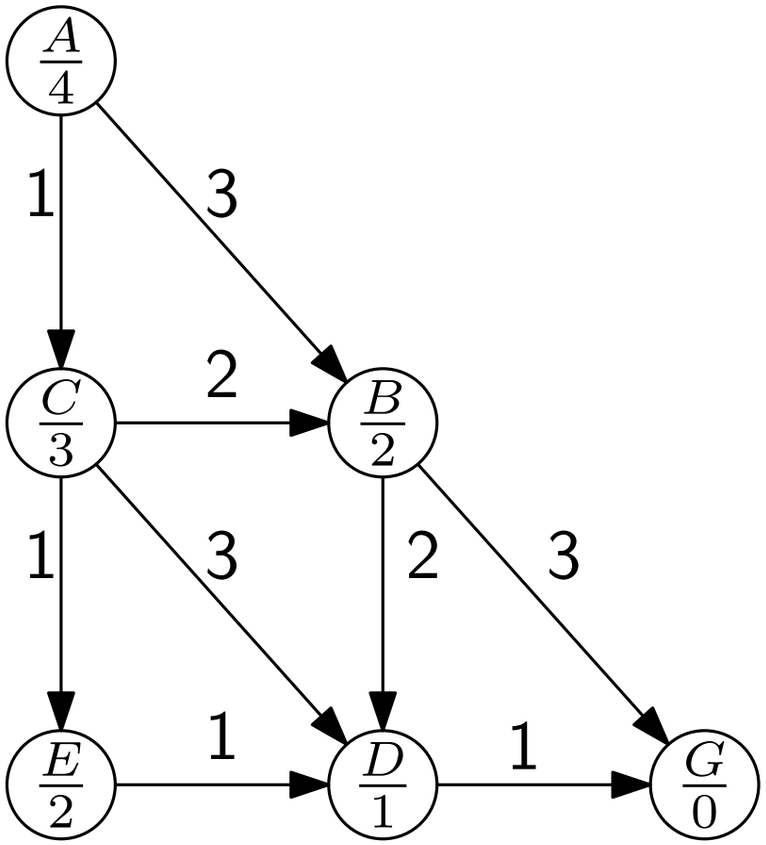
E	F
	A_0
$-A_0$	AB_3, AC_1

UCS Trace



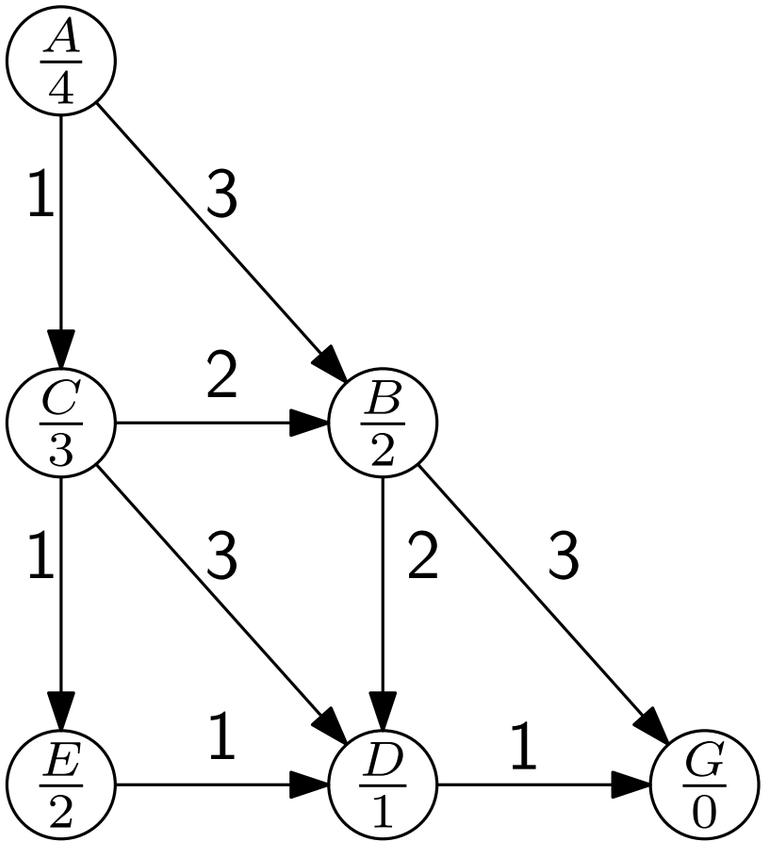
E	F
	A_0
$-A_0$	AB_3, AC_1
AC_1	AB_3, $C\cancel{B}_3$, CD_4, CE_2

UCS Trace



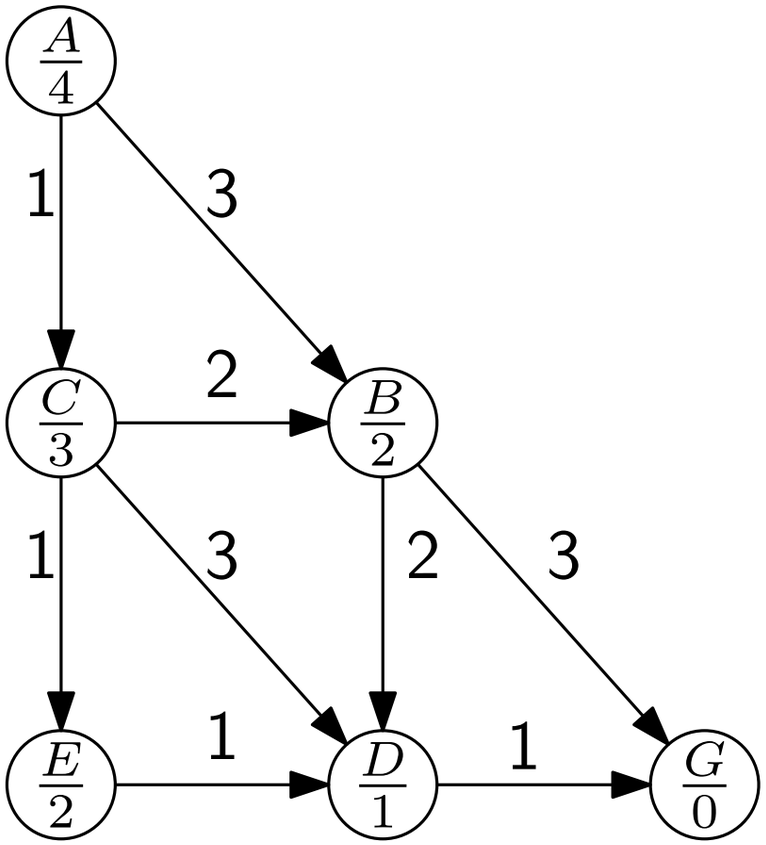
E	F
	A_0
$-A_0$	AB_3, AC_1
AC_1	AB_3, $C\cancel{B_3}$, CD_4, CE_2
CE_2	AB_3, $C\cancel{D_4}$, ED_3

UCS Trace



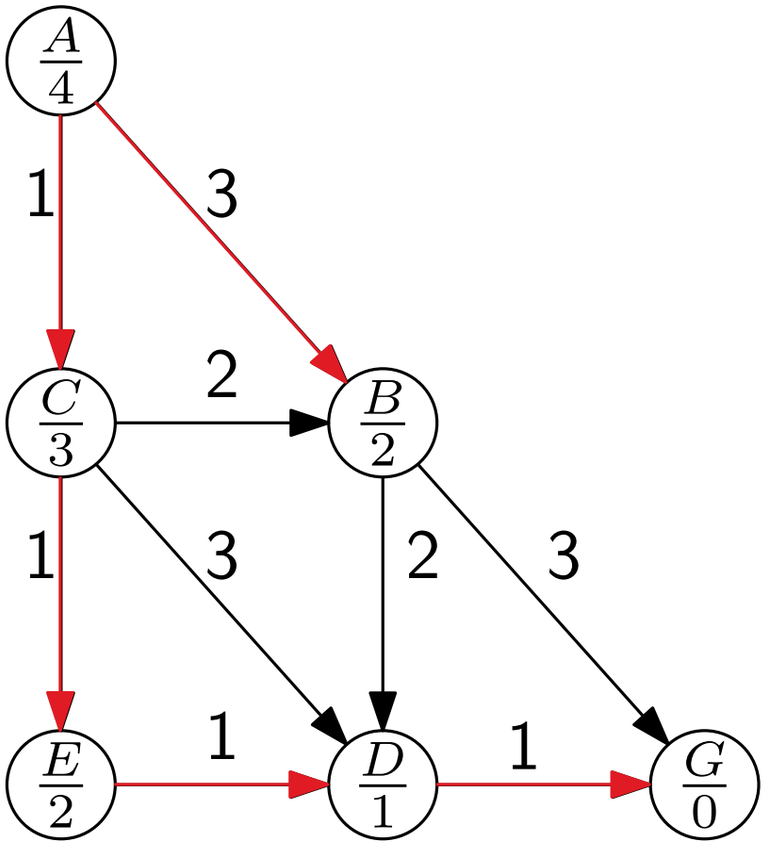
E	F
	A_0
$-A_0$	AB_3, AC_1
AC_1	AB_3, $C\cancel{B}_3$, CD_4, CE_2
CE_2	AB_3, $C\cancel{D}_4$, ED_3
AB_3	ED_3, $B\cancel{D}_5$, BG_6

UCS Trace



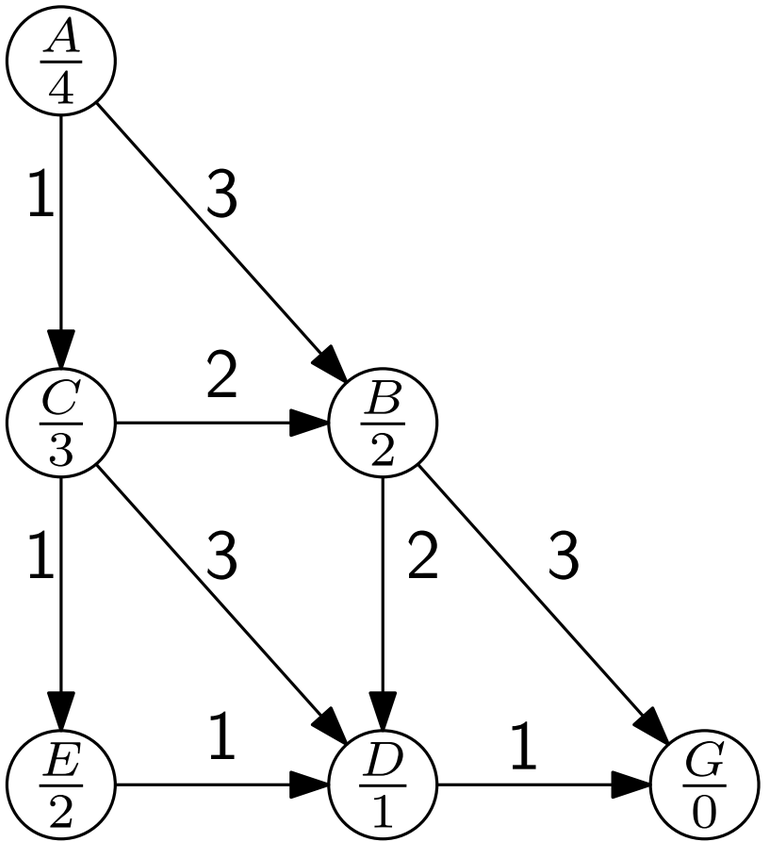
E	F
	A_0
$-A_0$	AB_3, AC_1
AC_1	AB_3, $C\cancel{B_3}$, CD_4, CE_2
CE_2	AB_3, $C\cancel{D_4}$, ED_3
AB_3	ED_3, $B\cancel{D_5}$, BG_6
ED_3	$B\cancel{G_6}$, DG_4

UCS Trace



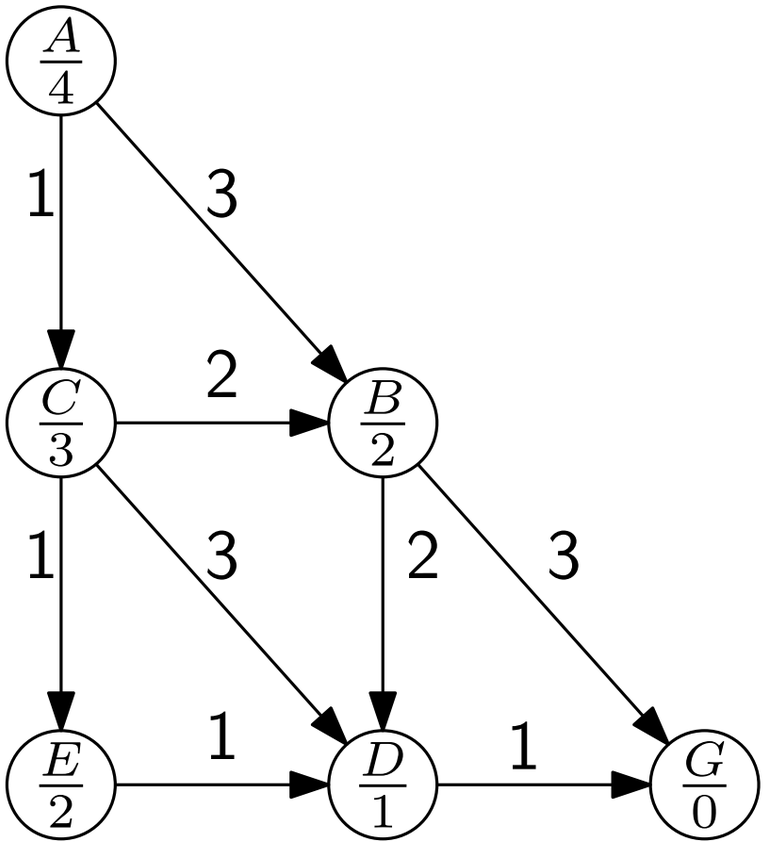
E	F
	A_0
$-A_0$	AB_3, AC_1
AC_1	AB_3, $\cancel{CB_3}$, CD_4, CE_2
CE_2	AB_3, $\cancel{CD_4}$, ED_3
AB_3	ED_3, $\cancel{BD_5}$, BG_6
ED_3	$\cancel{BG_6}$, DG_4
DG_4	

A* Trace



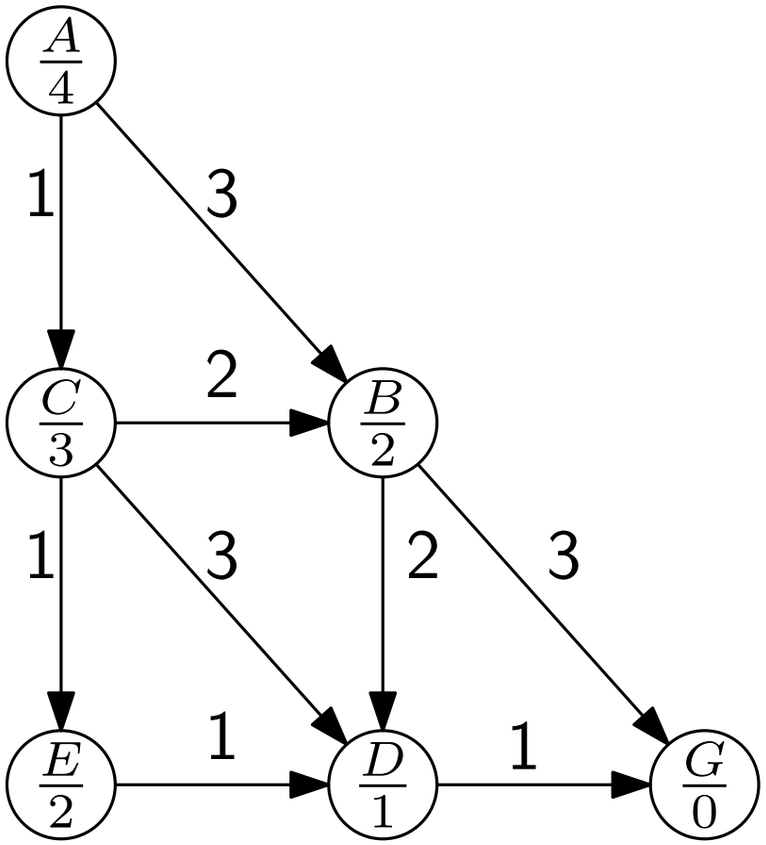
E	F
	—A0/4

A* Trace



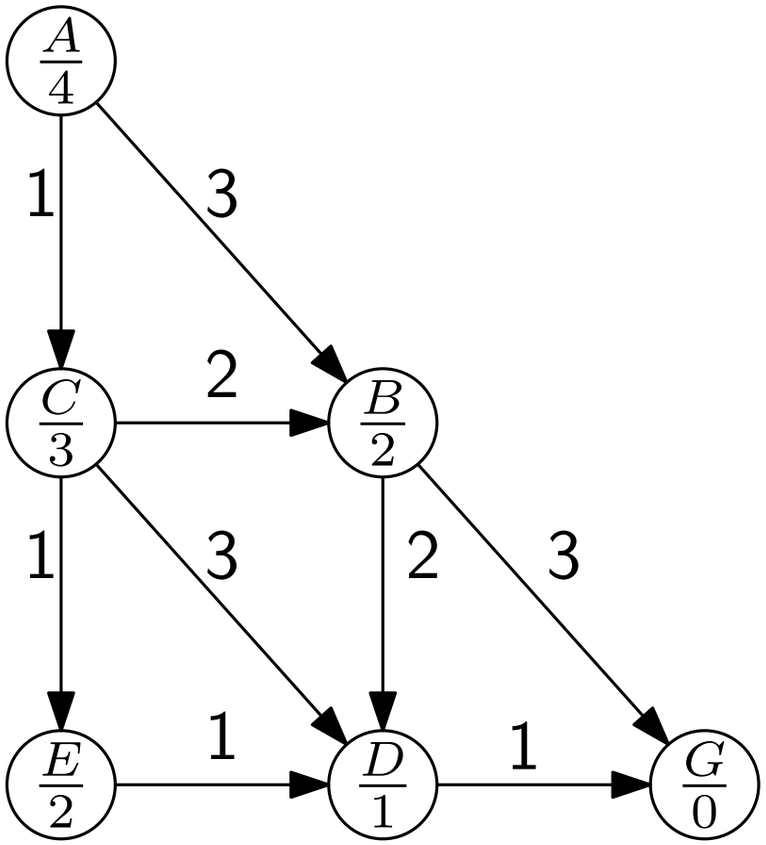
E	F
	$-A_{0/4}$
$-A_{0/4}$	$AB_{3/2}$, $AC_{1/3}$

A* Trace



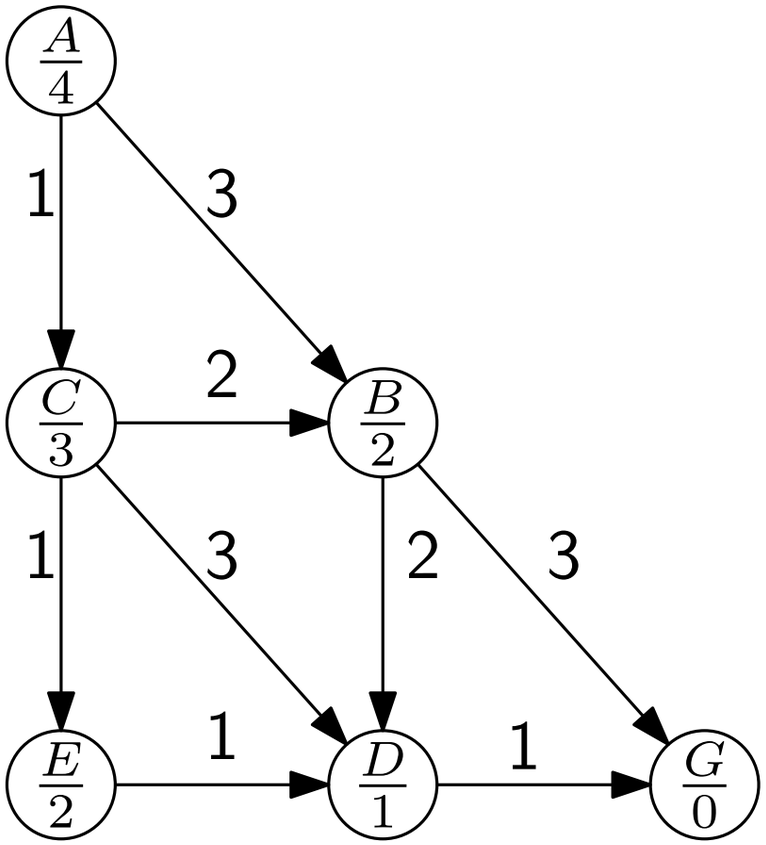
E	F
	$-A_{0/4}$
$-A_{0/4}$	$AB_{3/2}$, $AC_{1/3}$
$AC_{1/3}$	$AB_{3/2}$, $\cancel{CB_{3/2}}$, $CD_{4/1}$, $CE_{2/2}$

A* Trace



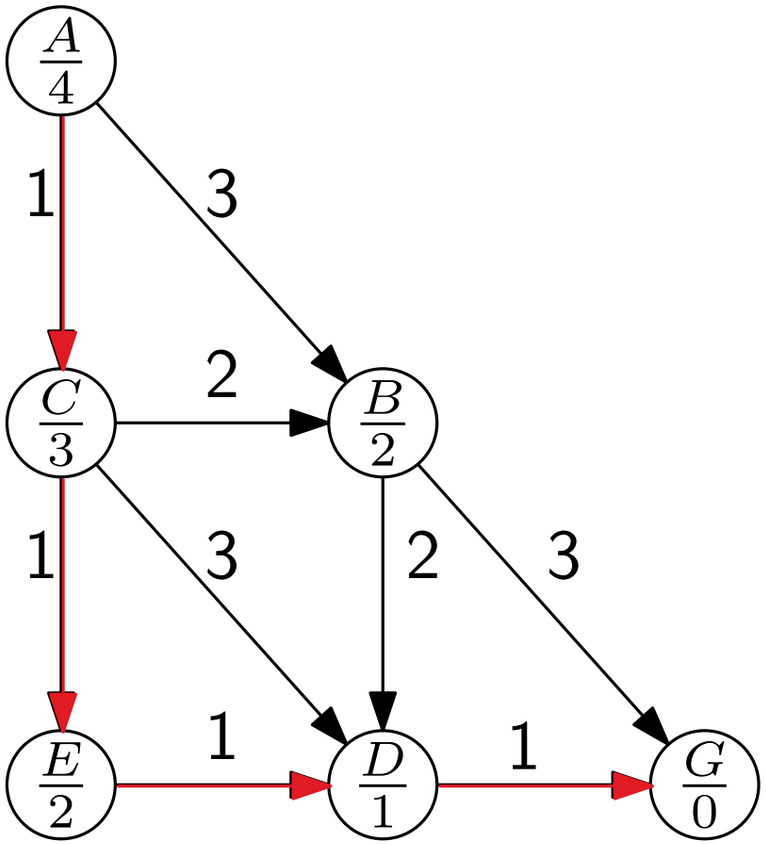
E	F
	$-A_{0/4}$
$-A_{0/4}$	$AB_{3/2}$, $AC_{1/3}$
$AC_{1/3}$	$AB_{3/2}$, $\cancel{CB_{3/2}}$, $CD_{4/1}$, $CE_{2/2}$
$CE_{2/2}$	$AB_{3/2}$, $\cancel{CD_{4/1}}$, $ED_{3/1}$

A* Trace



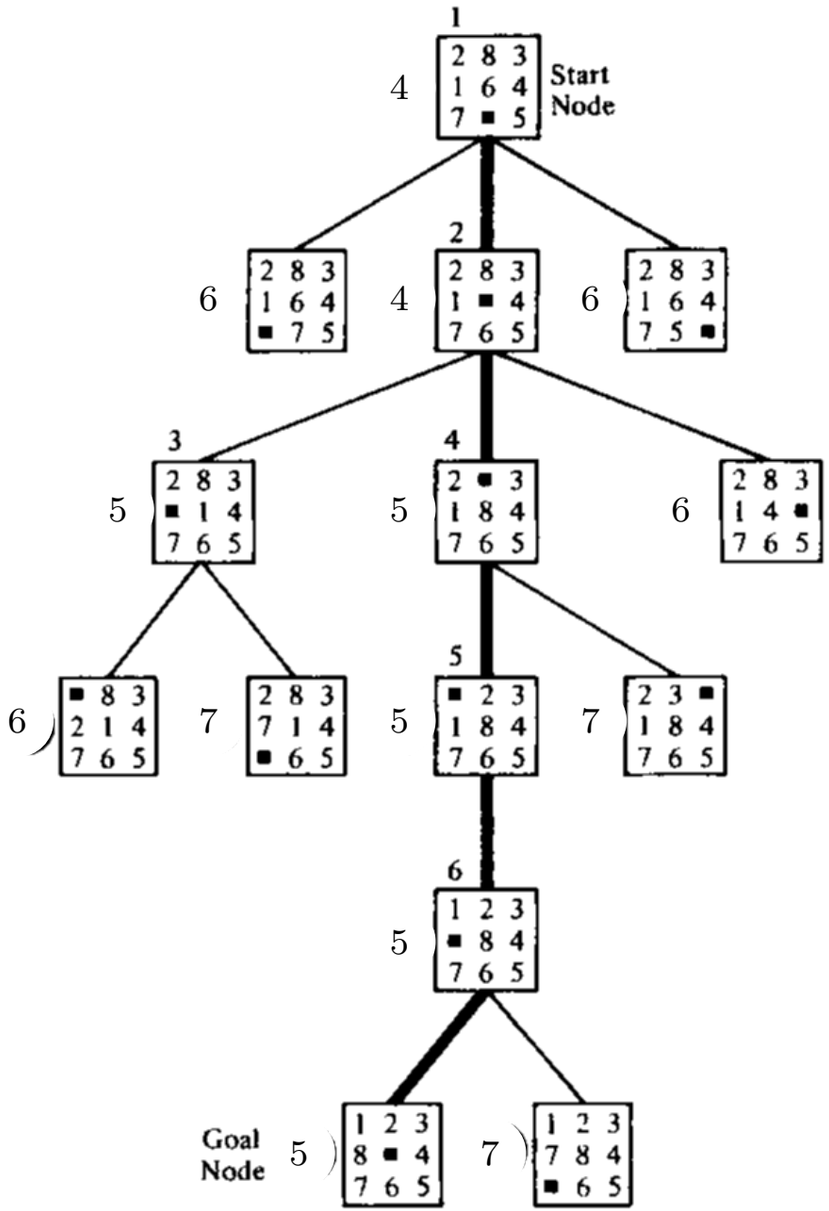
E	F
	$-A_{0/4}$
$-A_{0/4}$	$AB_{3/2}$, $AC_{1/3}$
$AC_{1/3}$	$AB_{3/2}$, $\cancel{CB_{3/2}}$, $CD_{4/1}$, $CE_{2/2}$
$CE_{2/2}$	$AB_{3/2}$, $\cancel{CD_{4/1}}$, $ED_{3/1}$
$ED_{3/1}$	$AB_{3/2}$, $DG_{4/0}$

A* Trace



E	F
	$-A_{0/4}$
$-A_{0/4}$	$AB_{3/2}$, $AC_{1/3}$
$AC_{1/3}$	$AB_{3/2}$, $\cancel{CB_{3/2}}$, $CD_{4/1}$, $CE_{2/2}$
$CE_{2/2}$	$AB_{3/2}$, $\cancel{CD_{4/1}}$, $ED_{3/1}$
$ED_{3/1}$	$AB_{3/2}$, $DG_{4/0}$
$DG_{4/0}$	

A* on 8-Puzzle



1	2	3
8	-	4
7	6	5

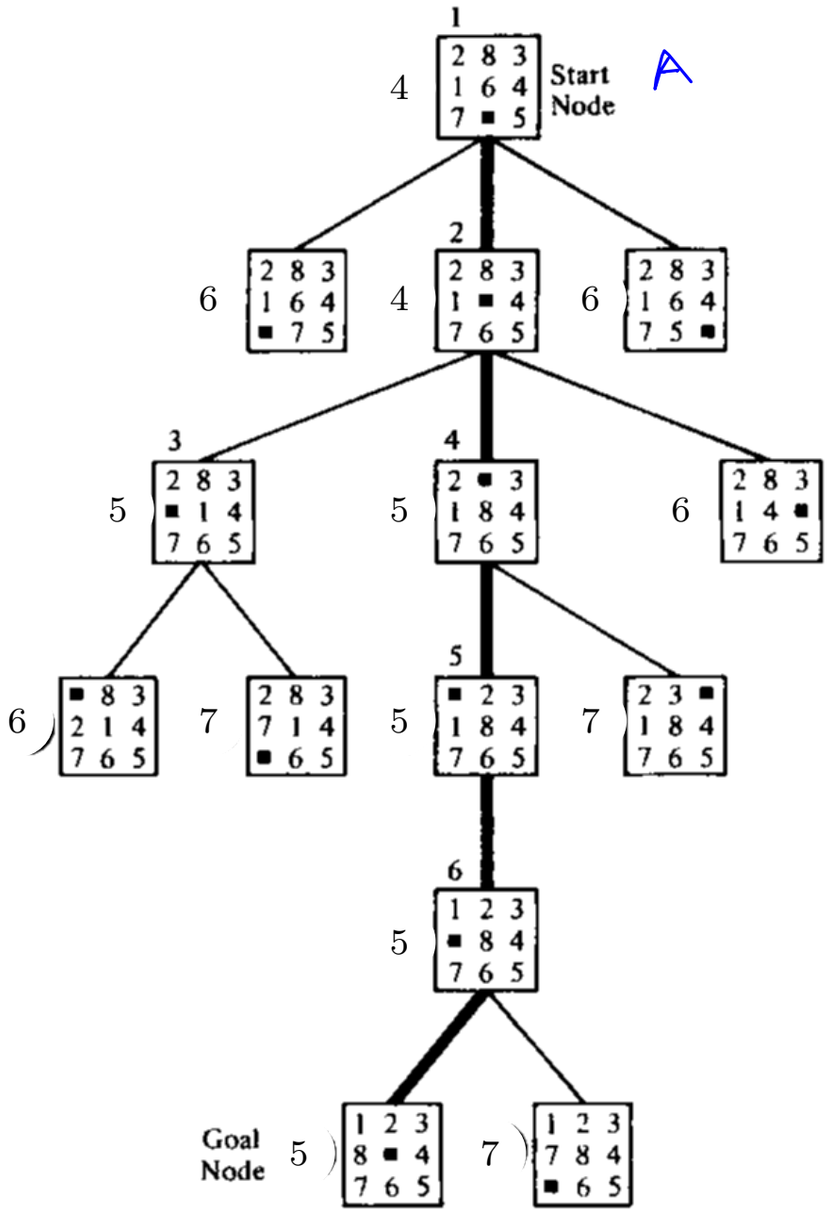
Goal state

2	8	3
1	6	4
7	-	5

Start state

$h(n)$ = number of misplaced tiles

A* on 8-Puzzle

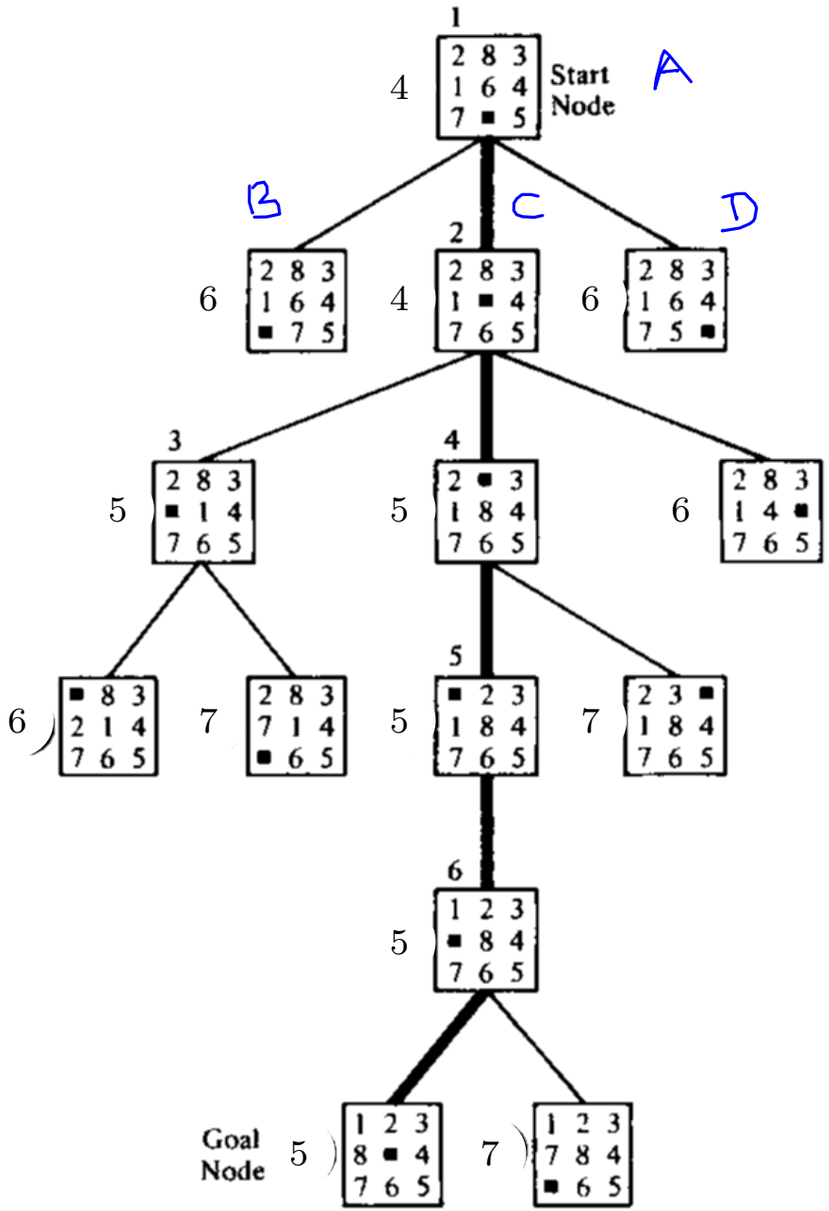


E

F

$-A_4$

A* on 8-Puzzle



E

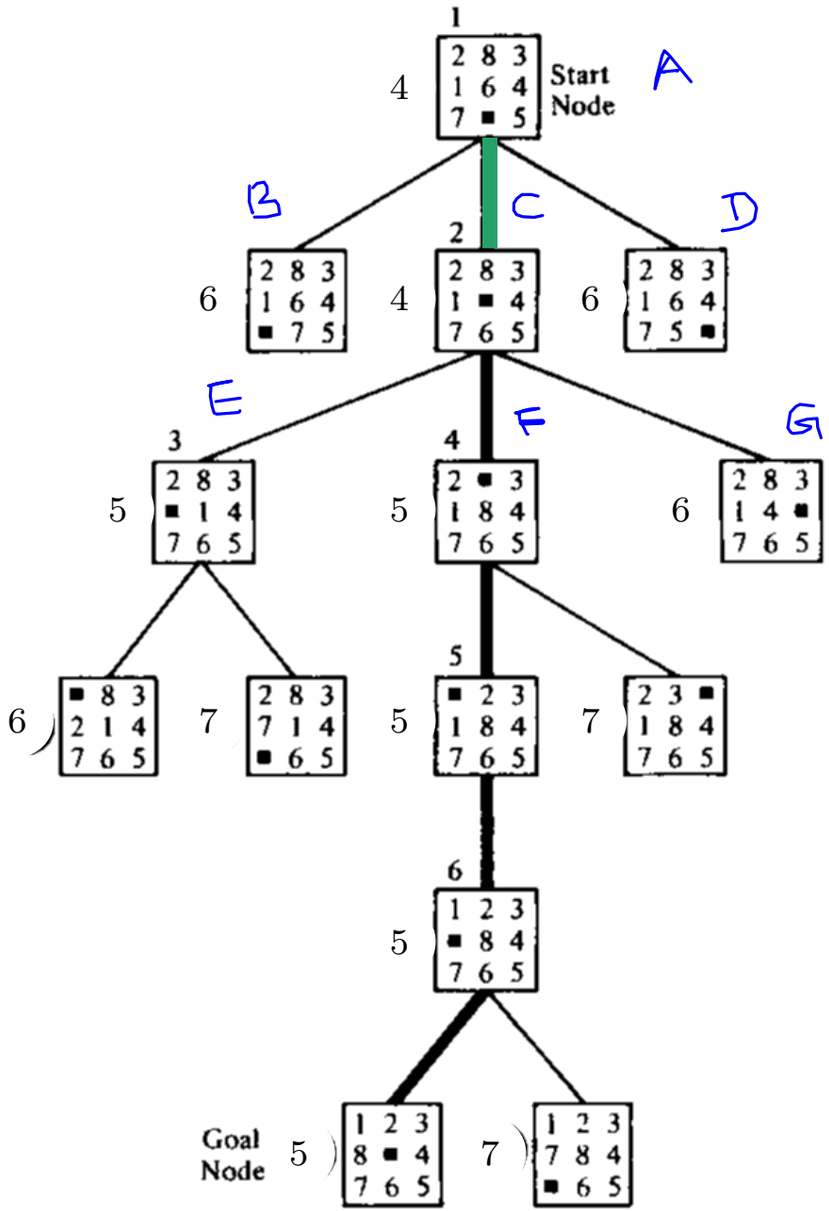
$-A_4$

F

$-A_4$

$AB_{1/5}$ $AC_{1/3}$ $AD_{1/5}$

A* on 8-Puzzle



E

$-A_4$

$AC_{1/3}$

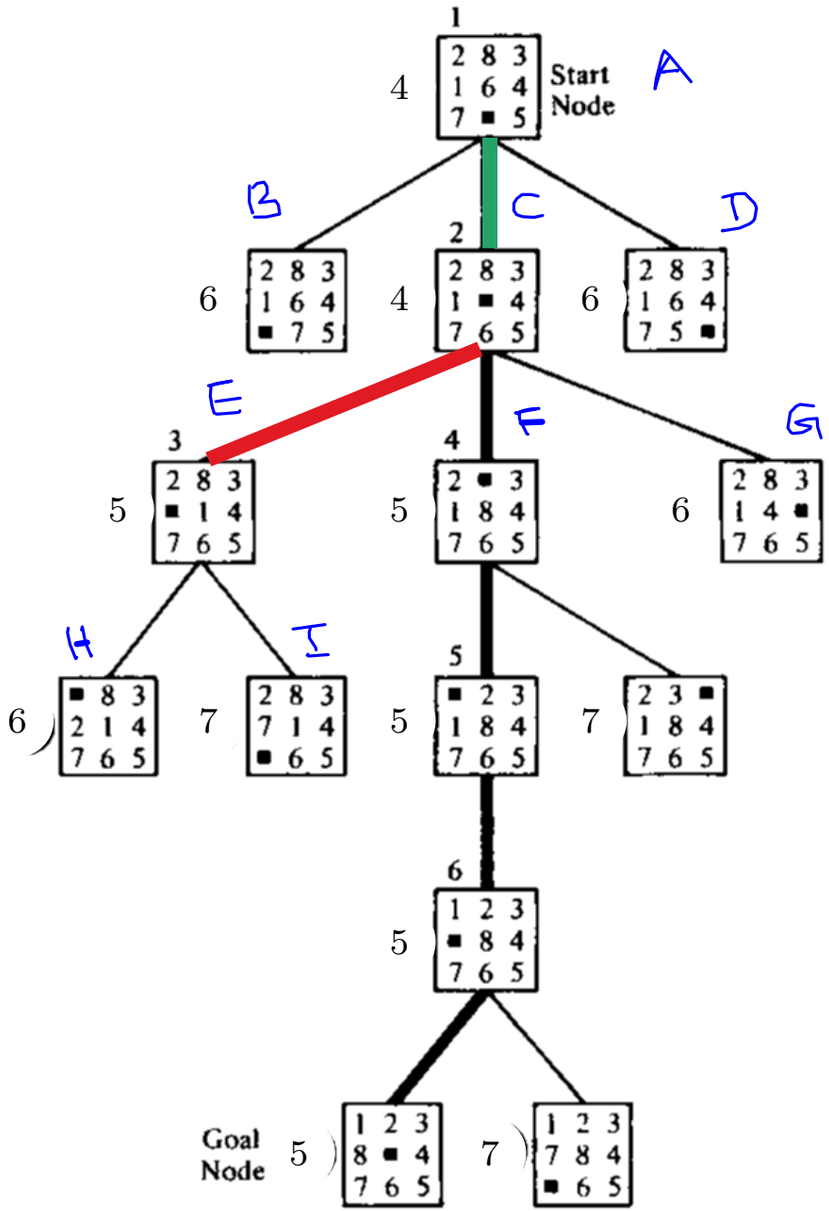
F

$-A_4$

$AB_{1/5}$ $AC_{1/3}$ $AD_{1/5}$

$AB_{1/3}$ $AD_{1/5}$ $CE_{2/3}$ $CF_{2/3}$ $CG_{2\ 4}$

A* on 8-Puzzle



E

$-A_4$

$AC_{1/3}$

$CE_{2/3}$

F

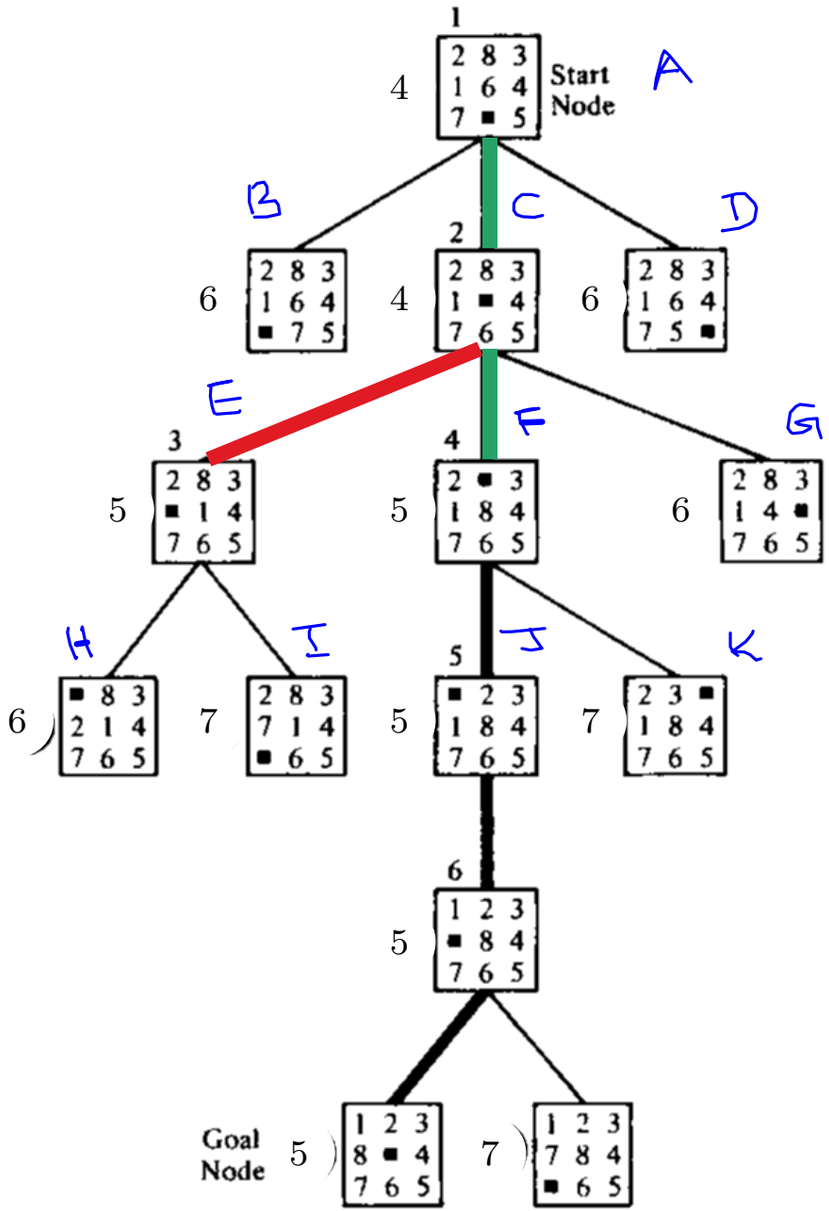
$-A_4$

$AB_{1/5}$ $AC_{1/3}$ $AD_{1/5}$

$AB_{1/3}$ $AD_{1/5}$ $CE_{2/3}$ $CF_{2/3}$ $CG_{2\ 4}$

$AB_{1/3}$ $AD_{1/5}$ $CF_{2/3}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$

A* on 8-Puzzle



E

$-A_4$

$AC_{1/3}$

$CE_{2/3}$

$CF_{2/3}$

F

$-A_4$

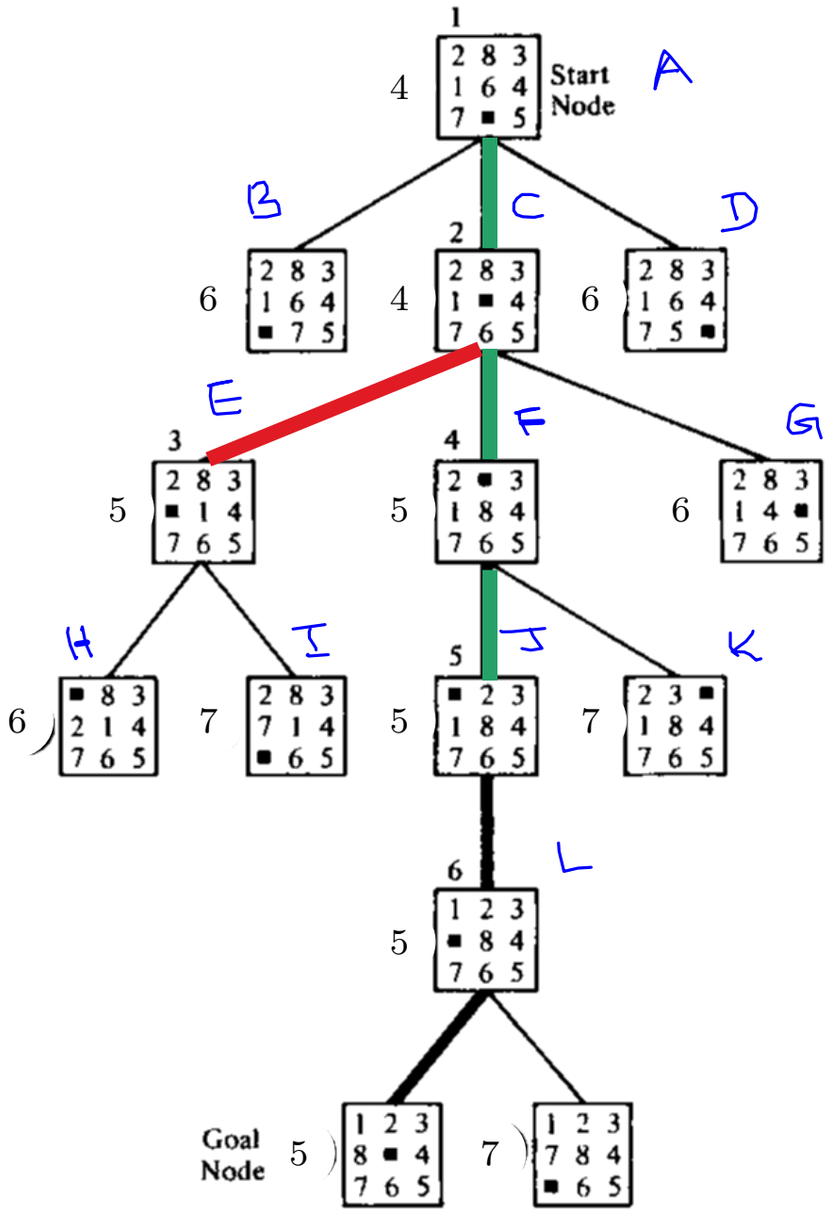
$AB_{1/5}$ $AC_{1/3}$ $AD_{1/5}$

$AB_{1/3}$ $AD_{1/5}$ $CE_{2/3}$ $CF_{2/3}$ $CG_{2\,4}$

$AB_{1/3}$ $AD_{1/5}$ $CF_{2/3}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$

$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FJ_{3/2}$ $FK_{3/4}$

A* on 8-Puzzle



E

$-A_4$

$AC_{1/3}$

$CE_{2/3}$

$CF_{2/3}$

$FJ_{3/2}$

F

$-A_4$

$AB_{1/5}$ $AC_{1/3}$ $AD_{1/5}$

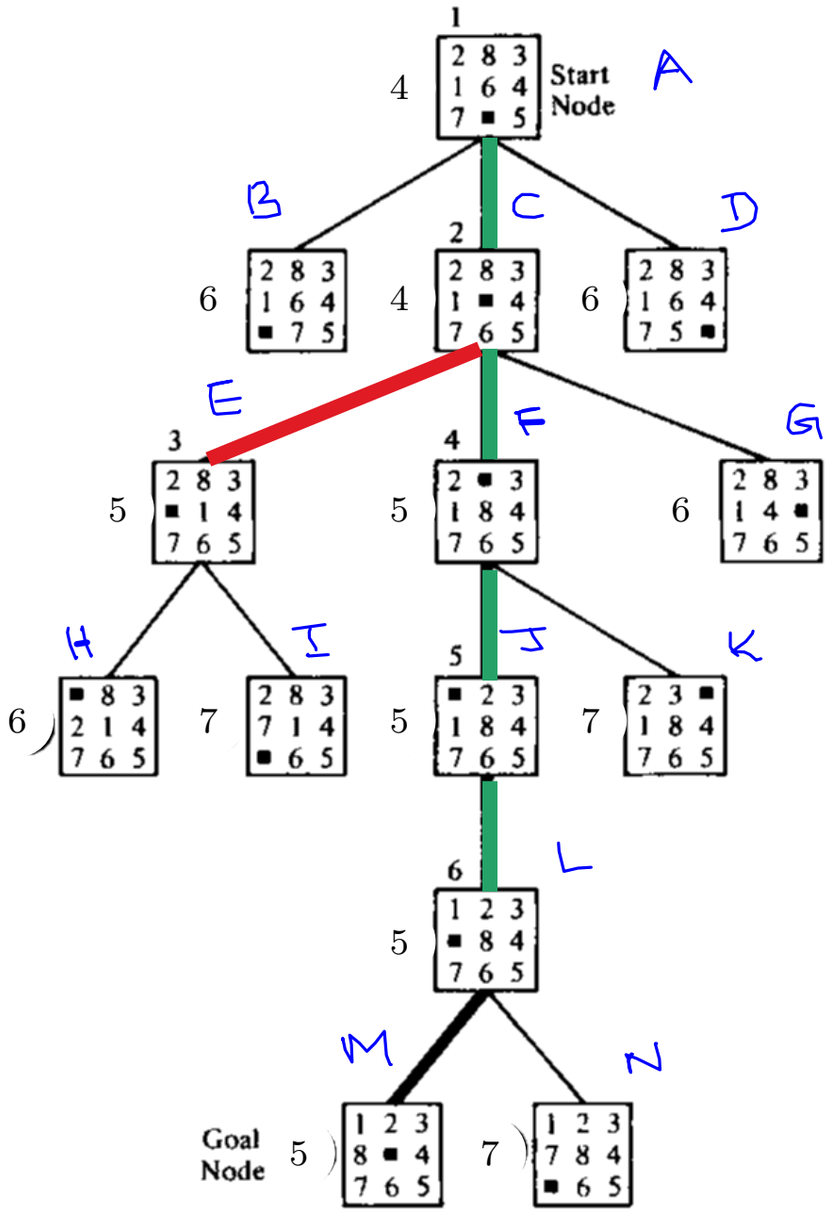
$AB_{1/3}$ $AD_{1/5}$ $CE_{2/3}$ $CF_{2/3}$ $CG_{2\,4}$

$AB_{1/3}$ $AD_{1/5}$ $CF_{2/3}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$

$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FJ_{3/2}$ $FK_{3/4}$

$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FK_{3/4}$ $JL_{4/1}$

A* on 8-Puzzle



E

$-A_4$

$AC_{1/3}$

$CE_{2/3}$

$CF_{2/3}$

$FJ_{3/2}$

$JL_{4/1}$

F

$-A_4$

$AB_{1/5}$ $AC_{1/3}$ $AD_{1/5}$

$AB_{1/3}$ $AD_{1/5}$ $CE_{2/3}$ $CF_{2/3}$ $CG_{2\,4}$

$AB_{1/3}$ $AD_{1/5}$ $CF_{2/3}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$

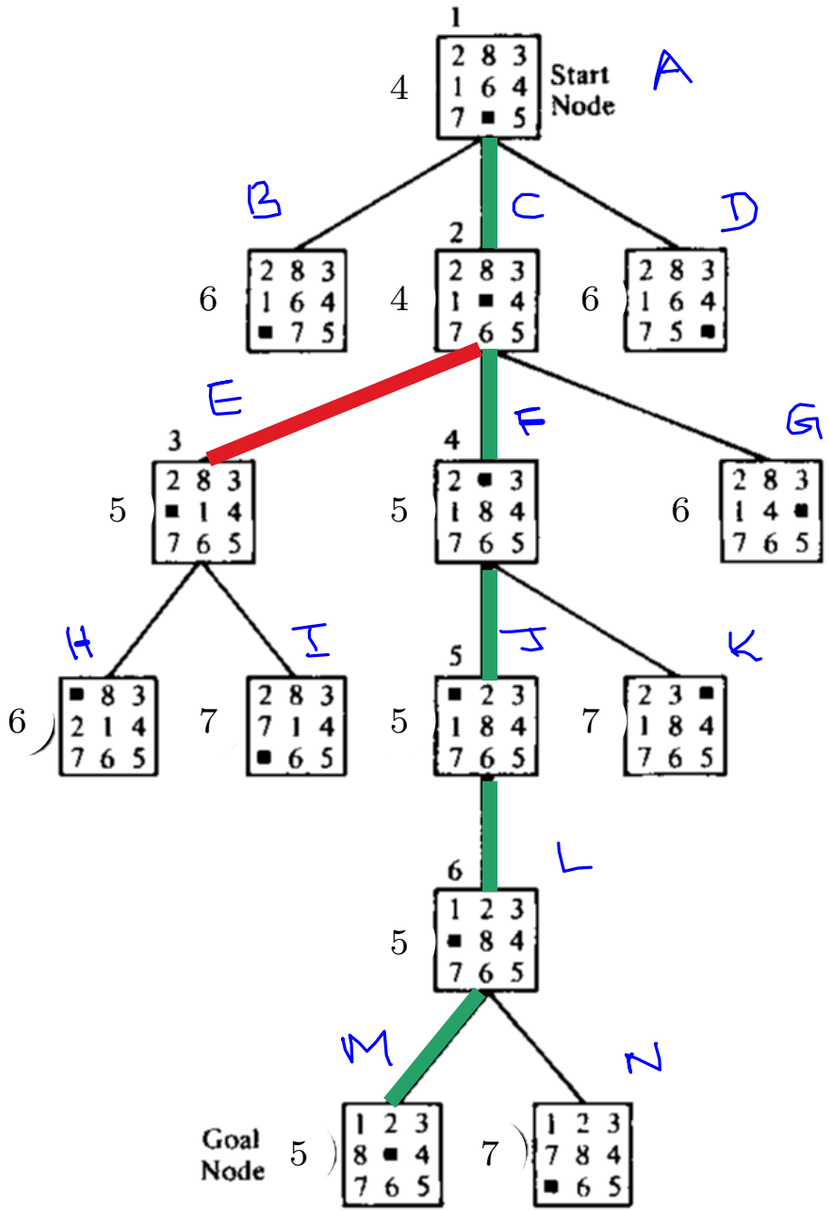
$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FJ_{3/2}$ $FK_{3/4}$

$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FK_{3/4}$ $JL_{4/1}$

$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FK_{3/4}$ $LM_{5/0}$

$LN_{5/2}$

A* on 8-Puzzle



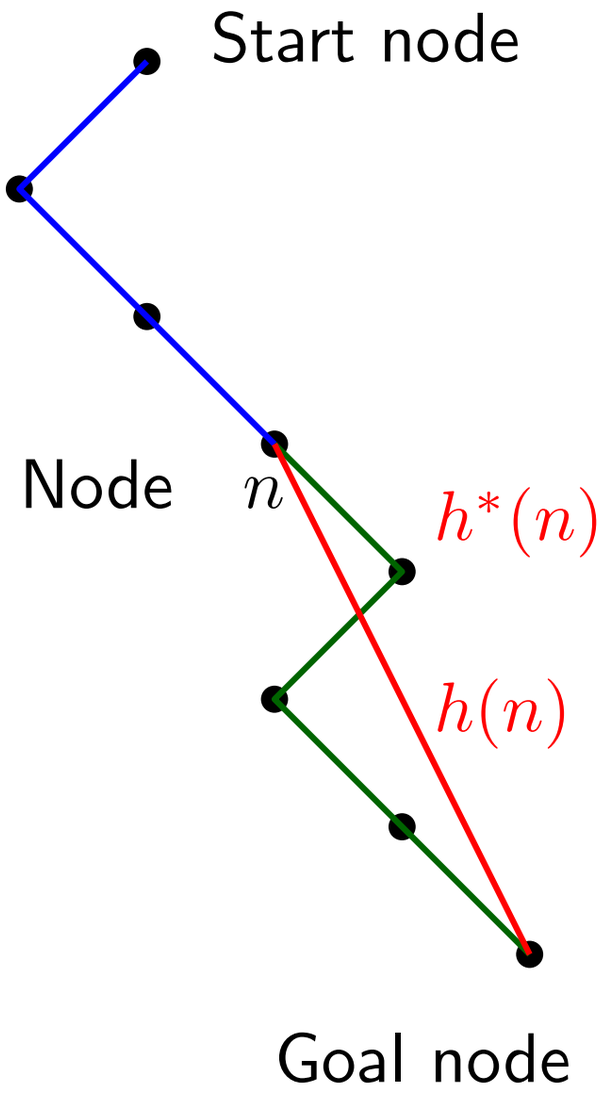
E	F
$-A_4$	$-A_4$
$AC_{1/3}$	$AB_{1/5}$ $AC_{1/3}$ $AD_{1/5}$
$CE_{2/3}$	$AB_{1/3}$ $AD_{1/5}$ $CE_{2/3}$ $CF_{2/3}$ $CG_{2/4}$
$CF_{2/3}$	$AB_{1/3}$ $AD_{1/5}$ $CF_{2/3}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$
$FJ_{3/2}$	$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FJ_{3/2}$ $FK_{3/4}$
$JL_{4/1}$	$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FK_{3/4}$ $JL_{4/1}$
$LM_{5/0}$	$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FK_{3/4}$ $LM_{5/0}$ $LN_{5/2}$
	$AB_{1/3}$ $AD_{1/5}$ $CG_{2/4}$ $EH_{3/3}$ $EI_{3/4}$ $FK_{3/4}$ $LN_{5/2}$

Adimissible Heuristics

- A heuristic function h is admissible (optimistic) if

$$0 \leq h(n) \leq h^*(n)$$

  where $h^*(n)$ is the true shortest distance to a nearest goal.
- $h(n)$ is admissible iff it underestimates the shortest distance from n to the goal.
- $f(n) = g(n) + h(n)$ underestimates the shortest distance from start state to goal via n
- Example: Euclidean distance
- A* in practice: Find admissible heuristics!

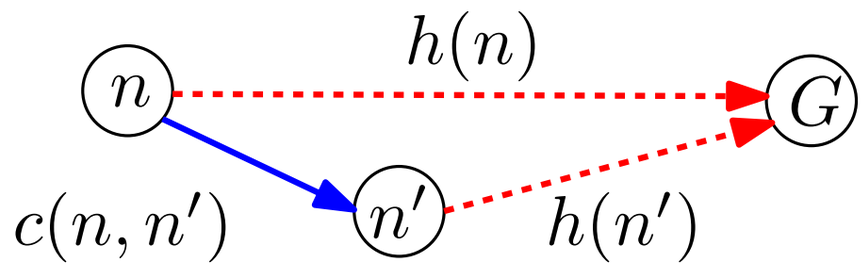


Consistent Heuristics

- Heuristic function h is consistent (monotone) if it satisfies triangle inequality

$$h(n) \leq c(n, n') + h(n') \text{ for every edge } (n, n')$$

- $h(n) - h(n') \leq c(n, n')$
- $h(t) = 0$ where t is a goal node.
- If h consistent, A* finds a shortest path to a goal.
- Consistent $\Rightarrow$ admissible



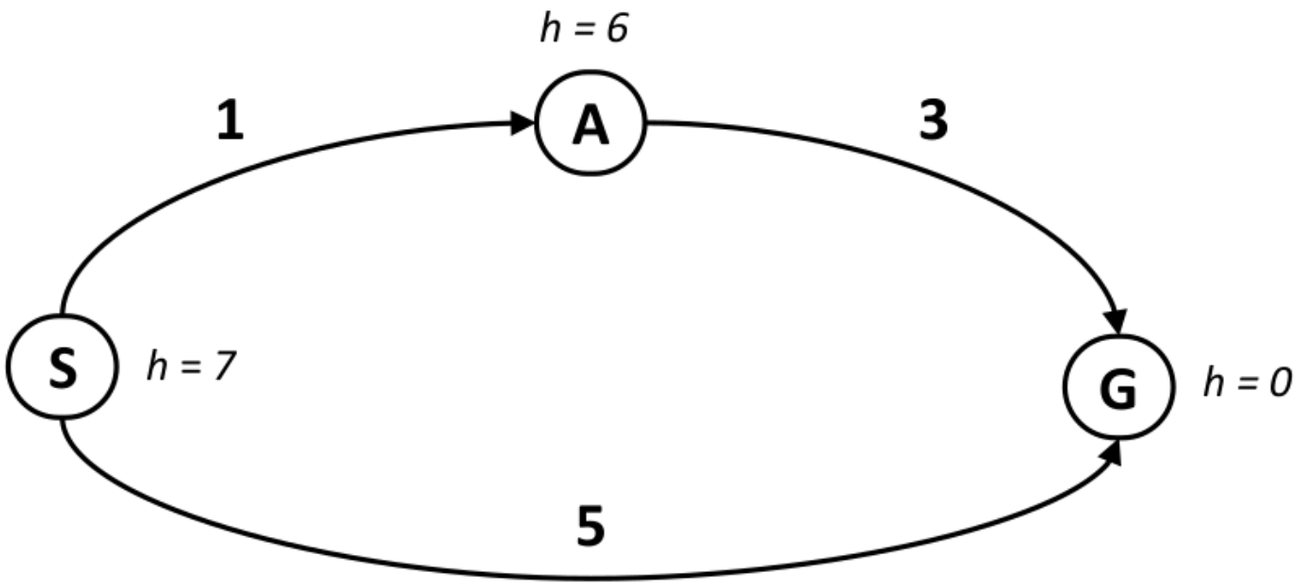
Conditions for Optimality

If there is a solution, A* always finds an optimal solution – the first path to a goal selected – if

- Branching factor b is finite (every node has a finite number of successors)
- All edge costs $> \epsilon$ (bounded) for some $\epsilon > 0$ (non-negative)
- Heuristic is admissible and consistent

What will happen if

- estimated goal distance $>$ actual distance?

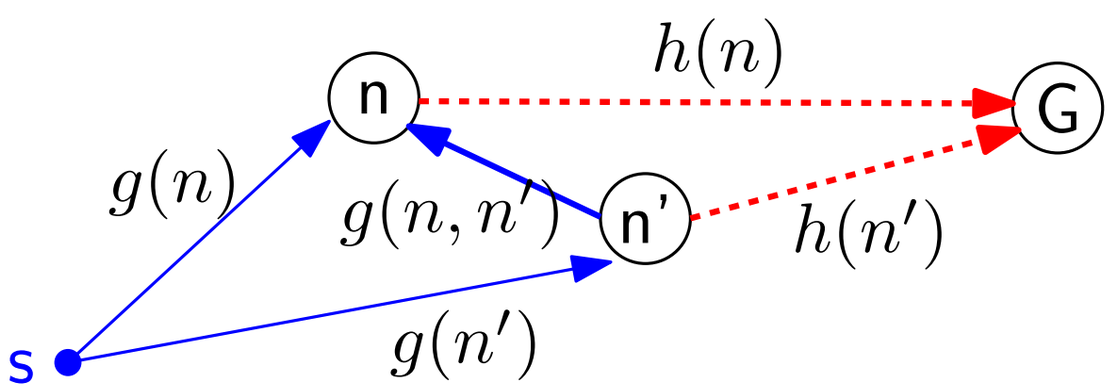


Optimality of A*

- Finds the shortest path to a goal
- Tree-search version of A* is optimal if $h(n)$ is admissible
- Graph-search version of A* is optimal if $h(n)$ is consistent.
- Correctness proof similar to PFS
- Use f, instead of g.

Correctness of A*

1. A* selected frontier node n.
2. Suppose there is a frontier node n' through which there is a shorter path to n. Let $g(n', n)$ be the shortest distance from n' to n.
   $g(n') + g(n', n) < g(n) \equiv g(n', n) < g(n) - g(n')$
3. A* selected n before n', so
   $g(n) + h(n) \leq g(n') + h(n') \equiv g(n) - g(n') < h(n') - h(n)$
4. From (2) and (3), $g(n', n) < h(n') - h(n)$
5. However, $h(n')$ is consistent $\equiv h(n') - h(n) \leq g(n', n)$.
6. Therefore, there is no shorter frontier path to n.



Time Complexity

- Number of nodes explored before exploring the goal node.
- UCS explores nodes in order of $g(v)$ (distance from start node) so that it will explore every node with distance $g(v)$ shorter than $g(t)$, the distance of the goal node, $g(v) \leq g(t)$.
- A* explores all nodes whose $f(v) = g(v) + h(v)$ is smaller than $f(t) = g(t) + h(t)$ where t is the goal node.
- $g(v) + h(v) \leq g(t) \equiv g(v) \leq g(t) - h(v)$
- Larger the $h(v)$, the fewer the nodes that satisfy this threshold.
- But $h(v)$ should still be admissible.

A* explores all nodes v satisfying

$$g(v) + h(v) \leq g(t)$$

$$g(v) \leq g(t) - h(v)$$

Heuristics

Heuristics by Relaxation: Overview

- Larger $h(v)$, the better the saving in time.
- From $h(v) = 0$ (UCS) to $h(v) = h^*(v)$ (difficult to compute).
- Remove constraints
  - easier search
  - closed form solution
  - independent subproblems
- Combine heuristics using max

Easier Search

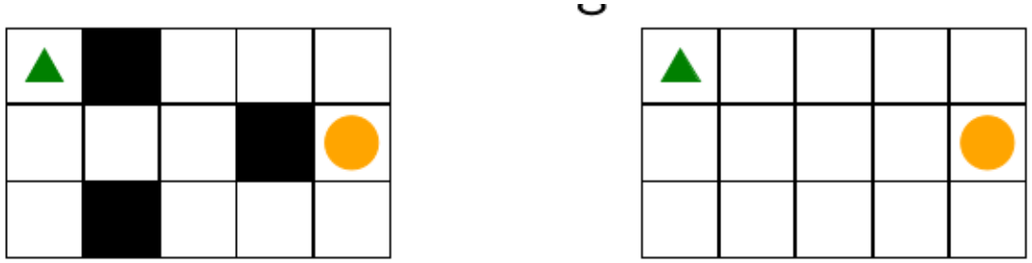
- Remove constraints $\Rightarrow$ Easier search
- A relaxation P_{relax} of a search problem P has distances that satisfy:

$$g_{\text{relax}}(u, v) \leq g(u, v)$$

- Given a relaxed search problem P_{relax} , define the heuristic $h(u) = g_{\text{relax}}(u, t)$, the shortest distance from u to goal node using $g_{\text{relax}}(u, v)$.
- Construct the shortest distances $h(u) = g_{\text{relax}}(u, t)$ to goal using DP.

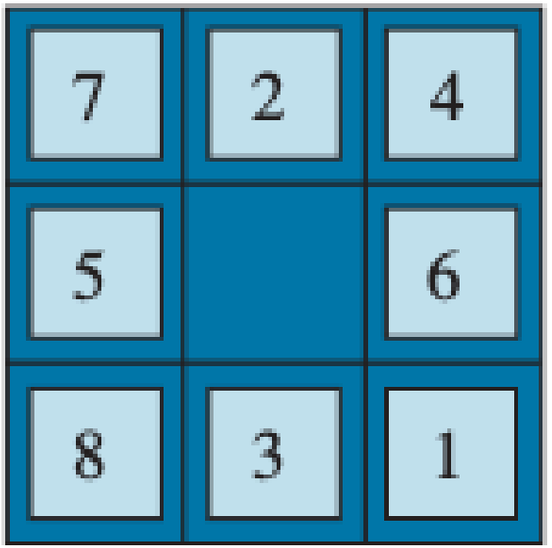
Closed Form Solution

- States are positions (r, c) on the grid. Possible actions: move up, down, left, or right, provided they don't move you into a wall or off the grid; all edge costs are 1.
- Goal: move from triangle to circle
- Start state is at (1, 1), and goal state is (2, 5).
- With an arbitrary configuration of walls, we can't compute $g(v, t)$ except by doing search.
- Relax the original problem by removing the walls, then we can compute $g(v, t)$ in closed form: it's just the Manhattan distance between s and t.
- ManhattanDistance$((r1, c1), (r2, c2)) = |r1 - r2| + |c1 - c2|$.

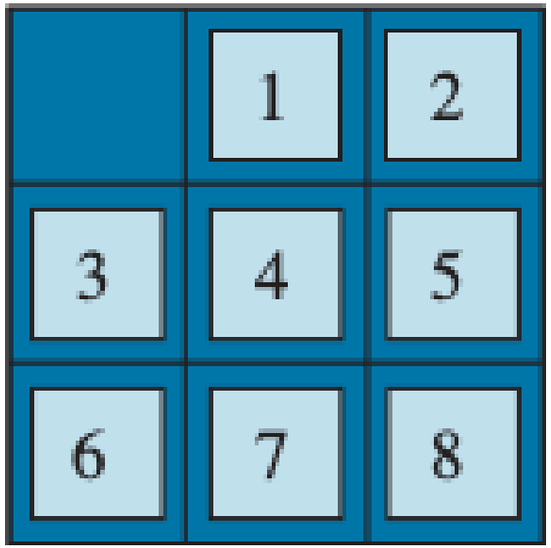


Independent Subproblems

- Original problem: tiles cannot overlap (constraint)
- Relaxed problem: tiles can overlap (no constraint) – any tile could slide any direction at any time, ignoring other tiles.
- Relaxed solution: decomposed into 8 independent problems – in some cases each solution in closed form



start state



goal state

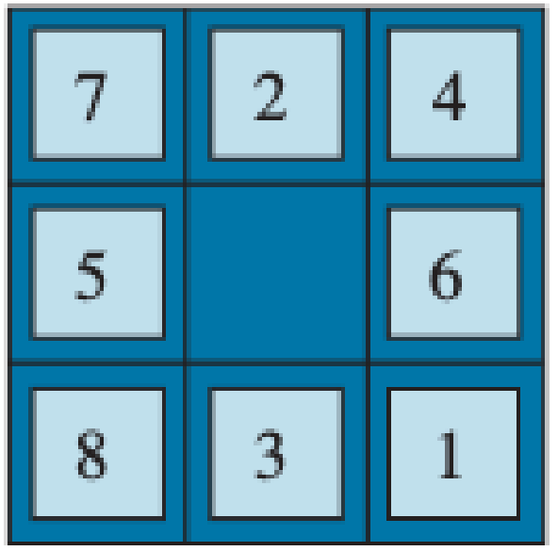
8-Puzzle has Exponential Number of States

- Branching factor ≈ 3
- Suppose goal state at depth, $d = 22$
- Tree search: $3^{22} = 3.1 \times 10^{10}$ states
- Graph search: $9!/2 = 181,440$ states

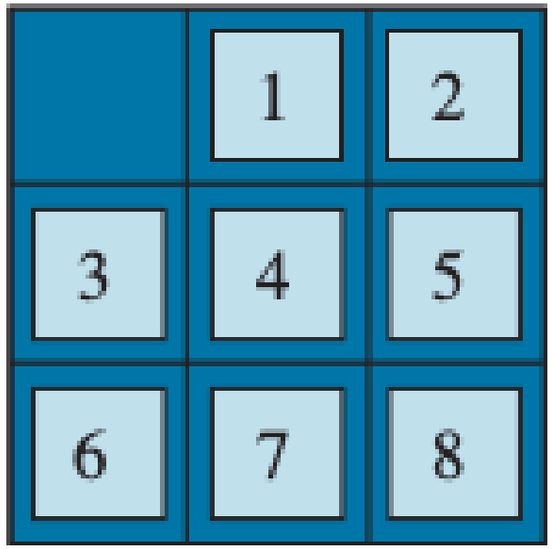
8-Puzzle Heuristic I

- Ideal: $h(v) = g(v, t)$, as hard as solving the problem.
- $h(v) =$ Number of misplaced tiles:
  start state, $h(v) = 8$; goal state, $h(t) = 0$
- Average nodes expanded when the optimal path has

	4 steps	8 steps	12 steps
UCS	112	6300	3.6×10^6
Tiles	13	39	227



start state

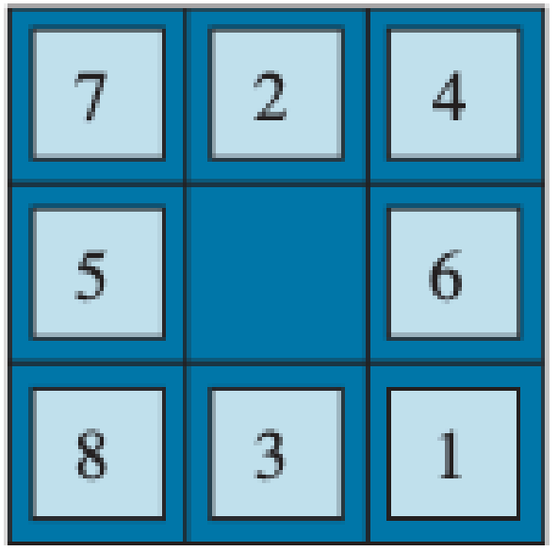


goal state

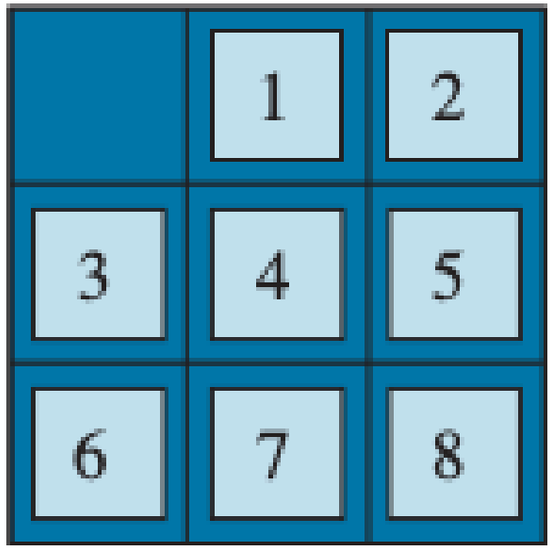
8-Puzzle Heuristic II

- $h(v) =$ Manhattan distance
  $h(v) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$
- Average nodes expanded when the optimal path has

	4 steps	8 steps	12 steps
Tiles	13	39	227
Manhattan	12	25	73



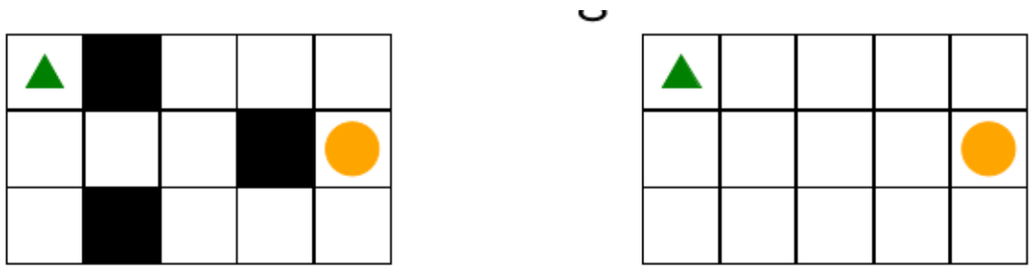
start state



goal state

General Framework

- Removing constraints
- Adding edges
  - Reducing edge lengths from ∞



- Original: $g((1,1), \text{East}) = \infty$
- Relaxed: $g_{\text{relax}}((1,1), \text{East}) = 1$

General Framework

- Relaxed search problem
  A relaxation P_{relax} of a search problem P has distances that satisfy:

$$g_{\text{relax}}(u, v) \leq g(u, v)$$

- Relaxed heuristic
  Given a relaxed search problem P_{relax} , define the relaxed heuristic $h(u) = h^*_{\text{relax}}(u)$, the shortest distance from u to goal node using $g_{\text{relax}}(u, v)$.
- $h^*_{\text{relax}}(u)$ should be easy/efficient to compute.

General Framework

- Consistency of relaxed heuristic
  Suppose $h(u) = h^*_{\text{relax}}(u)$ for some relaxed problem. Then $h(u)$ is consistent.
- Proof:
  - $h^*_{\text{relax}}(u) \leq g_{\text{relax}}(u, v) + h^*_{\text{relax}}(v)$ for an arc uv (triangle inequality)
  - $h(u) \leq g_{\text{relax}}(u, v) + h(v)$ for an arc uv ($h(u) = h^*_{\text{relax}}(u)$)
  - $h(u) \leq g(u, v) + h(v)$ (relaxation)
- If the problem is relaxed, consistency is guaranteed.

Tradeoff

- Efficiency:
  - $h(u) = h^*_{\text{relax}}(u)$ must be easy to compute
  - Closed form, easier search, independent subproblems
  - If $h(u) = 0$, then A* is same as UCS.
- Tightness:
  - Heuristic $h(u)$ should be close to $h^*(u)$
  - Don't remove too many constraints
  - If $h(u) = h^*(u)$, expands fewer nodes but does more work per node to compute $h(u)$.

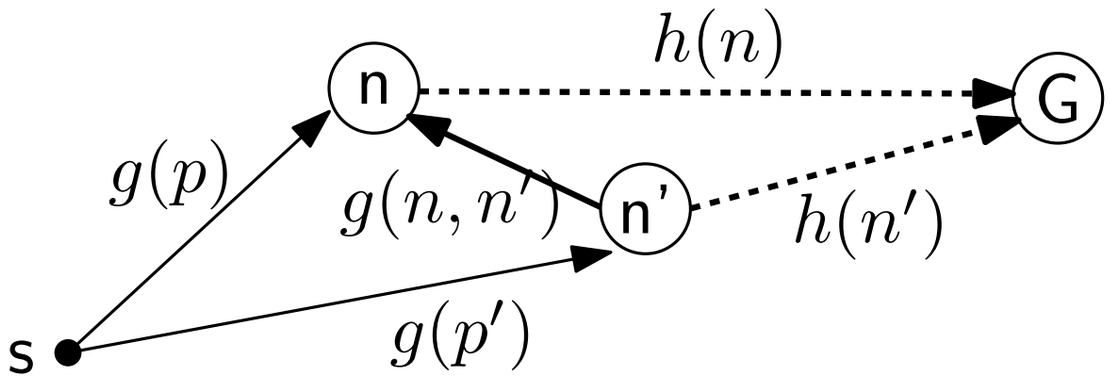
Max of Two Heuristics

- How do we combine two heuristics?
- Proposition: max heuristic
  - Suppose $h_1(u)$ and $h_2(u)$ are consistent.
  - Then $h(u) = \max h_1(u), h_2(u)$ is consistent.

End

Trade-off in Heuristic

- If $h(v) = 0$, then A* is same as UCS.
- If the true distance is used as a heuristic, $h(v) = g(v, t)$? A* only explores nodes on a shortest path from start to goal.
  - As heuristics get closer to the true distance, we will expand fewer nodes but usually do more work per node to compute the heuristic itself
- $h(v)$ is somewhere in between – a trade-off between quality of estimate and work per node.


$h(n)$
n
G
$g(p)$
$g(n, n')$
n'
$h(n')$
s
$g(p')$